

Reinforcement Learning for Energy Systems

Fundamentals and Applications

Elham Shirazi

e.shirazi@utwente.nl

University of Twente

PhD Seminar at Politecnico di Milano

23-April-2026

**UNIVERSITY
OF TWENTE.**



Why do we need learning-based methods in energy systems?

- Modern energy systems are becoming more distributed, dynamic, and uncertain.
- Their data is also becoming more complex, high-dimensional, noisy, heterogeneous, and time-dependent.
- In many such settings, classical white-box models remain essential, but are often no longer sufficient on their own.
- This creates a strong motivation for learning-based methods that can extract decision-relevant structure directly from data.

Control of energy systems

- charging or discharging now changes the future state of charge and later flexibility.
- shifting flexible loads now changes future demand.
- reactive power injection or tap changes now influence future network states.
- committing energy or flexibility in the energy market now creates future obligations, risks, and opportunities.

The control of energy systems is inherently a sequential decision-making problem, as present actions shape future system states, constraints, and operational objectives in a dynamic and uncertain environment.

What is Reinforcement Learning?

RL is a framework for optimizing sequential decisions under uncertainty, where actions have delayed consequences, learning requires balancing exploration and exploitation, and success depends on generalization.

When not to use RL?

RL is powerful, but it does not mean we should use it everywhere.

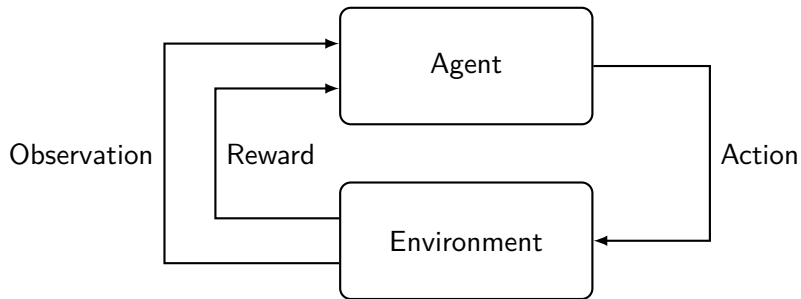
- The model is accurate and optimization is tractable.
- Safe exploration is impossible.
- The problem is one-shot rather than sequential.
- The available data is biased or non-representative.

Reinforcement Learning

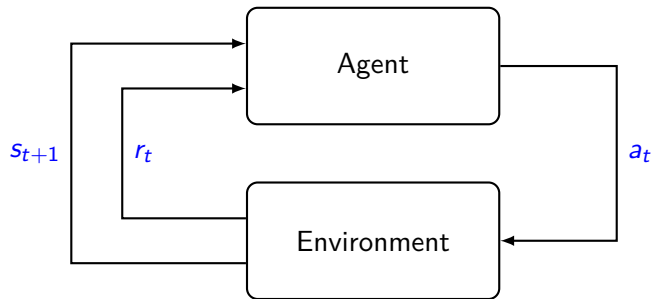
Reinforcement Learning involves

- Optimization
 - To find an optimal way to make decisions, yielding (almost) best outcomes.
- Delayed consequences
 - Reasoning about not just immediate benefit but also a longer term ramification
- Exploration
 - Learning about the world by making decisions. Decisions impact what we learn about
- Generalization
 - The ability to apply knowledge gained during training to new, unseen situations or environments.

Agent and environment



Agent and environment



Sequential decision making

SARSA

$$s_t, a_t, r_t, s_{t+1}$$

- Observe s_t
- Choose a_t
- Receive reward r_t
- Transition to s_{t+1}

A good action is defined by both immediate and future consequences.

History and state

- History $h_t = (o_1, a_1, r_1, \dots, o_t)$ contains everything observed so far.
- Agent chooses action a_t based on history
- State is a compact summary $s_t = f(h_t)$ for decision-making.
- A good state representation retains what matters.
- State design is one of the most important modeling choices in RL.

Markov Assumption

- State s_t is Markov if and only if

$$p(s_{t+1} \mid s_t, a_t) = p(s_{t+1} \mid h_t, a_t)$$

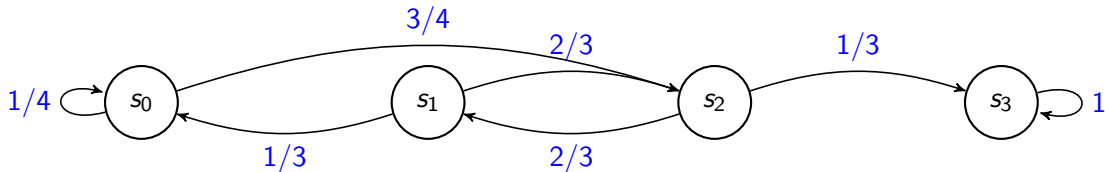
- Most recent observation is sufficient statistic of history $s_t = o_t$.
- Given s_t and a_t , earlier history no longer matters as present state summarizes the relevant past.

In energy systems, the Markov assumption is often only approximately true.

Markov Chain

A Markov chain describes a process that moves between states according to transition probabilities.

$$MC : (\mathcal{S}, \mathcal{P})$$



- $\mathcal{S}$: state space
- $\mathcal{P}(s_{t+1} = s' | s_t = s, a_t = a)$: transition model predict next agent states

Markov Reward Process (MRP)

A Markov reward process (MRP) extends a Markov chain by adding a reward associated with each state or transition, together with a discount factor.

$$MRP : (\mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma)$$

- $\mathcal{R}(s_t = s, a_t = a) = \mathbb{E}[r_t | s_t = s, a_t = a]$: reward model predicts immediate reward
- $\gamma \in [0, 1]$: discount factor

Markov Decision Process (MDP)

A Markov decision process (MDP) extends an MRP by introducing actions chosen by an agent.

$$MDP : (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

- $\mathcal{A}$: action space

Return and long-term reward

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- Return G_t is the discounted sum of future rewards.
- $\gamma = 0$: only immediate reward matters.
- Larger γ : future consequences matter more.
- In energy systems, this captures intertemporal coupling.

Policy

- Policy π defines how the agent acts.
- $\pi : \mathcal{S} \rightarrow \mathcal{A}$ mapping from states to actions
- Deterministic policy:

$$\pi(s) = a$$

you'll see μ later!

- Stochastic policy:

$$\pi(a | s) = \mathcal{P}(a_t = a | s_t = s)$$

Value functions

- The State Value Function $V(s)$ gives the long-term value of state s : how good it is to be in a state
- The Action Value Function $Q(s, a)$ gives the long-term value of state s , taking action a : how good it is to take an action in a state
- Value functions translate long-term effects into actionable quantities

Value functions

The State Value Function $V(s)$ is the expected return starting from s , ($V^\pi(s)$ is $V(s)$ following policy π):

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

The Action Value Function $Q(s, a)$ is the expected return starting from s , taking action a ($Q^\pi(s, a)$ is $Q(s, a)$ following policy π):

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Bellman Expectation equations

Bellman expectation: The value of a state or state-action pair equals the immediate reward plus the discounted expected value of what comes next under policy π

$$V^\pi(s) = \sum_a \pi(a|s) \left(\mathcal{R}(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right)$$

$$Q^\pi(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a')$$

- This recursive structure is the backbone of RL.
- It connects current choices to long-term performance.
- Most RL algorithms approximate Bellman-type relations.

Bellman Optimality equations

Bellman optimality: The optimal value of a state or state–action pair equals the immediate reward plus the discounted value of what comes next when the best possible future actions are chosen.

$$V^*(s) = \max_a \left(\mathcal{R}(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right)$$

$$Q^*(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

MDP Control

Compute the optimal policy

$$\pi^*(s) = \arg \max_{\pi} V^{\pi}(s)$$

- There exists a unique optimal value function
- Optimal policy for an MDP in an infinite horizon problem is deterministic and stationary.

Policy evaluation vs Control

- Policy evaluation: how good is a policy?
- Control: how do we find a better policy?

RL Terminology

- Model: Mathematical models of environment dynamics and reward
- Policy $\pi(a | s)$: A function that tells the agent which action to take in each state.
- Reward r : A scalar feedback indicating how good or bad an action outcome is.
- State $V^\pi(s)$ and action-state $Q^\pi(s, a)$ value function: future rewards from being in a state and/or action when following a particular policy.
- Return G_t : cumulative discounted future reward
- Trajectory: A sequence of states, actions, and rewards $\mathcal{T} = (s_0, a_0, r_1, s_0, a_1, r_2, \dots)$
- Episode: One complete interaction sequence from an initial to a terminal state.
- Exploration / Exploitation: trying new actions vs choosing the best-known one

RL Notation

Policy, reward, and return:

- $\pi(a | s)$: stochastic policy
- $\mu(s)$: deterministic policy
- $\gamma \in [0, 1]$: discount factor r_t : reward
- $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$: return

Value functions:

- $V^{\pi}(s)$: state-value function
- $Q^{\pi}(s, a)$: action-value function
- $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$: advantage function
- $\hat{A}_t$: estimated advantage
- $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$: TD error

RL Notation

Learning parameters:

- θ : actor / policy parameters
- ϕ : critic / value-function parameters
- α : learning rate (in SAC often entropy temperature)
- λ : trace-decay / GAE parameter
- τ : target-network update rate
- $\mathcal{D}$: replay buffer

Main families of RL

The main families of reinforcement learning are:

- 1 **Model-based RL**: the agent uses or learns a model of the environment dynamics and reward function, and then plans using that model.
- 2 **Model-free RL**: the agent learns directly from interaction with the environment, without an explicit model.
 - Value-based methods: learn a value function and derive the policy from it.
 - Policy-based methods: learn the policy directly.
 - Actor-Critic methods: combine a policy learner (actor) with a value estimator (critic).

Other RL classifications

Two other common ways to classify reinforcement learning methods are 1) Tabular methods vs. Function approximation, and 2) On-policy vs. Off-policy.

1 Tabular methods vs. function approximation:

- Tabular methods store a separate value for each state or state-action pair in a table, which is practical only when the state and action spaces are small and discrete.
- Function approximation methods use a parameterized model, such as a neural network, to estimate value functions or policies, which makes them suitable for large or continuous state and action spaces.

Other RL classifications

2 On-policy vs. Off-policy:

- On-policy methods learn using data is sampled from trajectories $\tau \sim \pi_\theta$ of same policy that is currently being improved. In other words, the agent learns from its own current behavior. e.g., REINFORCE, A2C, PPO.
- Off-policy Methods learn from data collected by a behavior policy different from the current target policy. These transitions are stored in a replay buffer $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1}, d_t)\}$ which allows past experience, including data generated by older or exploratory policies, to be reused during training. e.g., Q-learning, DQN, DDPG, TD3, SAC.

Model-free RL

Model-Free Reinforcement Learning

Value-Based Methods

Idea: Learn a value function and derive the policy by selecting the action with the highest estimated value.

Recall the two value functions:

- state value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

- action value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

The policy is obtained by choosing the action with the highest estimated value:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Value Estimate

The value function is estimated when an exact representation is infeasible or too complex, enabling generalization across similar states.

- 1 Sampling: Instead of computing exact expectations over all possible future trajectories, it learns from *sampled trajectories*,

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

- 2 Bootstrapping: means updating a value estimate using another current estimate of the next step instead of waiting for the full return. A standard example is using the TD target to update $V(s_t)$.
- 3 Value function approximation: value functions are represented by parameterized models,

$$V_{\phi}(s) \approx V^{\pi}(s), \quad Q_{\phi}(s, a) \approx Q^{\pi}(s, a).$$

Common Value-Based Methods

- If we use this $V_\phi(s) \approx V^\pi(s)$, then the approximation is "biased".
- If we use too many random future rewards like in G_t , then the total return has "high variance."

Progression of methods:

- 1 Temporal Difference (TD) learning: update mechanism for value estimation
- 2 Q-learning: extends TD learning to learn the optimal $Q_\phi(s, a)$ action-value function
- 3 Deep Q Network (DQN): deep neural network approximation of Q-learning

Temporal-Difference Learning

Temporal-Difference (TD) learning updates value estimates using observed rewards and current predictions.

For state-value estimation:

$$V(s_t) \leftarrow V(s_t) + \alpha \delta_t$$

δ_t is the TD error

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Key characteristics:

- learns directly from interaction data
- updates after every transition
- bootstraps from the current estimate of the next state

Q-Learning

Q-learning extends TD learning to learn the optimal action-value function:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

Q-learning:

- learns the optimal $Q^*(s, a)$
- derives a policy via $\pi^*(s) = \arg \max_a Q^*(s, a)$
- is model-free and off-policy
- works well in tabular, discrete action spaces.

Deep Q-Networks

Deep Q-Networks (DQN) combine Q-learning with **value function approximation**.

Instead of storing action values in a table, DQN learns a parameterized action-value function $Q(s, a; \theta)$ where θ denotes the neural network parameters.

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta) \right)^2 \right]$$

- tabular methods do not scale to large or high-dimensional state spaces
- many practical problems involve continuous observations
- function approximation generalizes across similar states
- DQN addresses this by replacing the Q-table with a neural network
- DQN can handle continuous state spaces, but the action is still discrete.

ϵ -Greedy Exploration

ϵ -greedy is a simple exploration strategy mainly used in value-based RL methods, such as Q-learning and DQN.

At each step, the agent selects

$$a_t = \begin{cases} \arg \max_a Q(s_t, a), & \text{with probability } 1 - \epsilon, \\ \text{random action}, & \text{with probability } \epsilon. \end{cases}$$

- $1 - \epsilon$: exploit the current best action
- ϵ : explore by trying a random action
- larger ϵ : more exploration
- smaller ϵ : more exploitation

Policy-Based Methods

Idea: Learn the policy directly, without first learning a value function to derive it.

The policy is parameterized as:

$$\pi_{\theta}(a|s)$$

The goal is to find policy parameters θ that maximize the expected return.

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_t]$$

Policy-based methods are attractive when the action space is **continuous** or when a stochastic policy is desired.

Policy Gradient

The key quantity is the gradient of the objective:

$$\nabla_{\theta} J(\theta)$$

Using the policy gradient theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi(a_t | s_t; \theta) Q^{\pi}(s_t, a_t)]$$

Interpretation:

- increase the probability of actions with high return
- decrease the probability of actions with low return
- optimize the policy by gradient ascent

REINFORCE

REINFORCE is a Monte Carlo policy gradient method that updates the policy parameters using sampled returns:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t | s_t; \theta) G_t$$

Key points:

- sample trajectories using the current policy and compute the return G_t from time step t
- reinforce the probability of actions that lead to high return
- does not require an explicit value function for action selection
- provides an unbiased estimate of the policy gradient
- suffers from high variance, which can make learning slow and unstable.

Actor-Critic Methods

Actor-Critic methods combine **policy learning** with **value estimation**.

Two components:

- 1 Actor: updates the policy $\pi(a | s; \theta)$ using the advantage $A^\pi(s, a)$ (selects the action).
- 2 Critic: estimates/approximates the value function to guide the actor (evaluates the value of that action).

Why it helps?

- reduces variance compared with REINFORCE, replace G_t with $A^\pi(s, a)$ or δ_t
- enables more efficient and stable learning, because critic replaces noisy full returns with a lower-variance value-based learning signal.

Common actor-critic methods

- 1 **A2C/A3C**: Advantage/Asynchronous Advantage Actor-Critic methods that use the critic to estimate value functions and stabilize policy updates.
- 2 **PPO**: Proximal Policy Optimization, which improves stability by limiting how much the policy can change at each update.
- 3 **DDPG**: Deep Deterministic Policy Gradient, designed for continuous action spaces with a deterministic actor and a critic estimating $Q(s, a)$.
- 4 **SAC**: Soft Actor-Critic, which encourages both reward maximization and exploration by adding an entropy term to the objective.

Value Function approximation

- **Sampling:** expectations are approximated from sampled trajectories,

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

- **Bootstrapping:** targets use current value estimates,

$$r_t + \gamma V(s_{t+1}).$$

- **Function approximation:** value functions are represented by parameterized models,

$$V_{\phi}(s) \approx V^{\pi}(s), \quad Q_{\phi}(s, a) \approx Q^{\pi}(s, a).$$

Advantage

The advantage function $A^\pi(s, a)$ measures how much better a selected action a under policy π is compared with the average value of the current state s .

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

- Relative quality of an action, $A^\pi(s, a) > 0$ action is better than expected.
- $A^\pi(s, a)$ is unknown because $Q^\pi(s, a)$ and $V^\pi(s)$ are unknown, so we use an estimate.
- $\hat{A}_t$ denotes an estimate/approximation of the advantage for the sampled state-action pair (s_t, a_t) . $\hat{A}_t \approx A^\pi(s_t, a_t)$

Advantage approximations

- 1 Temporal-difference (TD) error: The most common approximation of advantage is based on the TD error (δ_t), and mainly one-step TD approximations,

$$\delta_t = r_{t+1} + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

- 2 Monte Carlo (MC): Using the full sampled return G_t ,

$$\hat{A}_t^{\text{MC}} = G_t - V_{\phi}(s_t)$$

- 3 Generalized Advantage Estimation (GAE) is a weighted sum of future TD errors,

$$\hat{A}_t^{\text{GAE}}(\gamma, \lambda) = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

Which advantage approximation to use?

Remember the trade-off between *bias* and *variance*! In AC, λ is the trace-decay parameter, which balances the bias-variance trade-off

Bias-Variance trade-off

if $\lambda = 0 : GAE = TD$, and if $\lambda \rightarrow 1 : GAE \rightarrow MC$

Advantage Actor-Critic (A2C/A3C)

- 1 **Key idea:** the actor improves the policy, while the critic provides a lower-variance learning signal. Their main strength is stable and efficient on-policy learning.
- 2 **Actor:** updates the stochastic policy using the estimated advantage

$$\mathcal{L}_{\text{actor}}(\theta) = -\mathbb{E} \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad \hat{A}_t \approx \delta_t$$

- 3 **Critic:** learns the state-value function

$$\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E} \left[(R_t - V_{\phi}(s_t))^2 \right]$$

- A2C is the synchronous version: multiple workers collect experience and update jointly.
- A3C is the asynchronous version: workers update independently in parallel.

Proximal Policy Optimization (PPO)

- 1 **Key idea:** PPO stabilizes policy optimization by preventing excessively large policy updates.

- 2 **Actor:** updates the policy using a clipped surrogate objective

$$\mathcal{L}_{\text{actor}}(\theta) = -\mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

- 3 **Critic:** learns the state-value function

$$\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E} \left[(R_t - V_{\phi}(s_t))^2 \right]$$

- 4 Advantage estimate:

$$\hat{A}_t \approx \hat{A}_t^{\text{GAE}}(\gamma, \lambda)$$

Deep Deterministic Policy Gradient (DDPG)

- 1 **Key idea:** DDPG combines deterministic policy gradients with a Q-critic for a continuous action space.
- 2 **Actor:** learns a deterministic policy $\mu_\theta(s)$ It is trained to maximize the critic value:

$$\mathcal{L}_{\text{actor}}(\theta) = -\mathbb{E}_{s_t} [Q_\phi(s_t, \mu_\theta(s_t))]$$

- 3 **Critic:** learns the action-value function

$$\mathcal{L}_{\text{critic}}(\phi) = \mathbb{E} \left[(Q_\phi(s_t, a_t) - y_t)^2 \right]$$

DDPG is sensitive because the actor gradient depends directly on the critic estimate $Q_\phi(s, a)$. Therefore, critic's errors can be amplified by bootstrapping $y_t = r_t + \gamma Q_\phi(s_{t+1}, \mu_\theta(s_{t+1}))$, function approximation $Q_\phi(s, a) \approx Q^\mu(s, a)$, and deterministic policy updates $a_t = \mu_\theta(s_t)$. The actor chases critic's mistakes!!!

Soft Actor-Critic (SAC)

Key idea: SAC is an off-policy actor-critic method for continuous control that learns a stochastic policy and encourages exploration through entropy regularization.

- maximize reward while also keeping the policy sufficiently exploratory through entropy regularization
- use off-policy learning for high sample efficiency
- use two critics to improve stability and reduce overestimation
- robust exploration, stable learning, and high sample efficiency
- strong performance in continuous action spaces
- more complex and computationally heavier than simpler actor-critic methods
- requires tuning of entropy and optimization hyperparameters

SAC Critic

The critic learns two soft Q-functions:

$$Q_{\phi_1}(s_t, a_t), \quad Q_{\phi_2}(s_t, a_t).$$

The "soft" Bellman target:

$$y_t = r_t + \gamma \left(\min_{i=1,2} Q_{\phi'_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1} \mid s_{t+1}) \right),$$

with

$$a_{t+1} \sim \pi_{\theta}(\cdot \mid s_{t+1}).$$

Each critic is trained by minimizing the below, meaning it estimates expected return plus future entropy bonus.

$$L(\phi_i) = \mathbb{E} \left[(Q_{\phi_i}(s_t, a_t) - y_t)^2 \right], \quad i \in \{1, 2\}.$$

SAC Actor

The actor is a stochastic policy and outputs a distribution over actions instead of a single action:

$$a_t \sim \pi_{\theta}(\cdot \mid s_t).$$

It is optimized to maximize both expected return and policy entropy:

$$J(\pi) = \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))].$$

$$\mathcal{H}(\pi(\cdot \mid s)) = -\mathbb{E}_{a \sim \pi(\cdot \mid s)} [\log \pi(a \mid s)]$$

Here, $\mathcal{H}(\pi(\cdot \mid s_t))$ is the policy entropy, and $\alpha > 0$ is the temperature parameter that controls the tradeoff between exploitation (smaller values $[10^{-3}, 10^2]$) and exploration (larger values $\alpha > 10^{-1}$).

Summary of the four AC methods:

- A2C/A3C: on-policy actor-critic using TD error or advantage estimates.
- PPO: on-policy actor-critic with clipped policy updates for stability.
- DDPG: off-policy deterministic actor-critic for continuous actions.
- SAC: off-policy stochastic actor-critic with entropy regularization.

Reinforcement Learning in Energy Systems Application

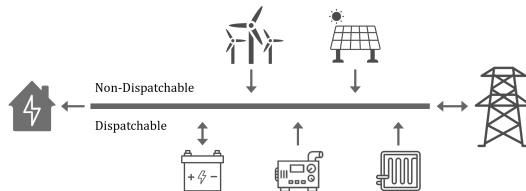
Where RL could appear in energy systems?

- 1 Microgrid Control
- 2 Battery Management Systems (BMS)
- 3 Power system operation

Case 1: Microgrid Control

Problem and case study

The MG consists of PV, WT (non-dispatchable), ESS, FC, and MT (dispatchable).

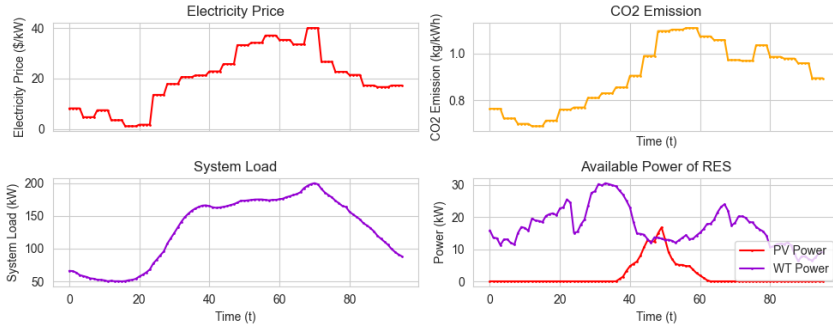


Control objective:

$$C_{\text{total}} = \lambda_{\text{oper}} C_{\text{oper}} + \lambda_{\text{env}} C_{\text{env}}$$

Case 1: Microgrid Control

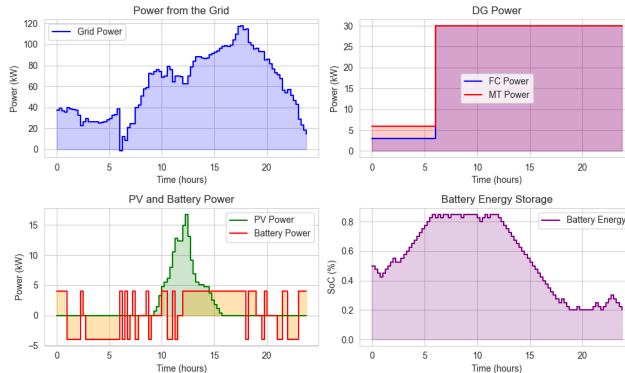
Data



Case 1: Microgrid Control

MILP

A twist to an old friend! To make it a bit more uncertain, Gaussian noise with varying intensity has been added to input power profiles.



Case 1: Microgrid Control

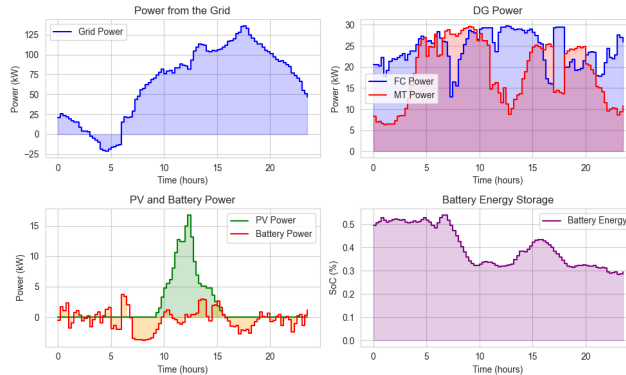
RL setup

- State $s_t = [t, P_L(t), P_{\text{Wind}}(t), P_{\text{PV}}(t), c_G(t), e_G(t), \text{SOC}(t)]$
- Actions $a_t = [P_{\text{ESS}}(t), P_{\text{DG},1}(t), \dots, P_{\text{DG},N}(t)]$
- Reward $r_t = \alpha - \beta C_{\text{total},t} - \gamma \Phi_t$

Case 1: Microgrid Control

AC-based RL

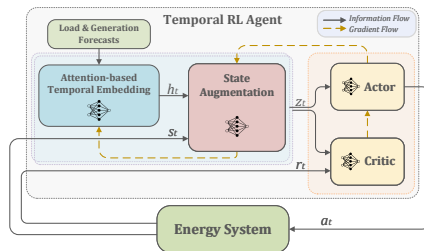
SAC!



Case 1: Microgrid Control

Temporal RL

While RL agents can learn system dynamics from historical data, they often struggle to capture temporal patterns, particularly in high-fluctuating conditions. Temporal RL agents can utilize forecasted patterns in the control of energy systems.



Temporal embedding

1 LSTM-based

- Forecast signals are converted into latent temporal features.
- The controller acts on an augmented state.

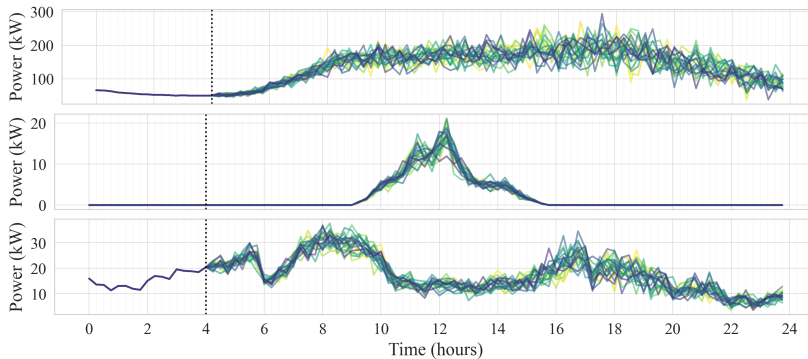
2 Attention-based

- Not all forecast intervals matter equally.
- Attention emphasizes the most decision-relevant parts of forecast.

Case 1: Microgrid Control

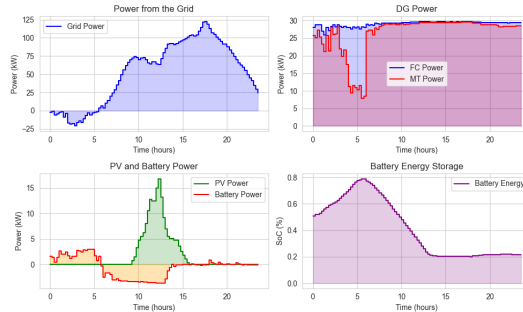
Temporal RL

Noisy forecasts fed to TRL



Case 1: Microgrid Control

LSTM Temporal RL



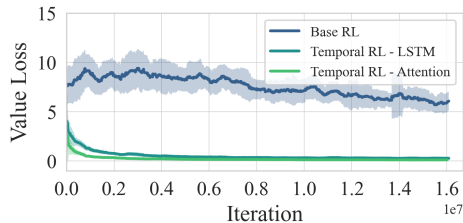
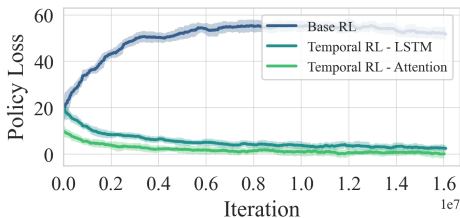
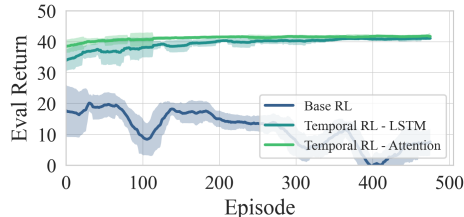
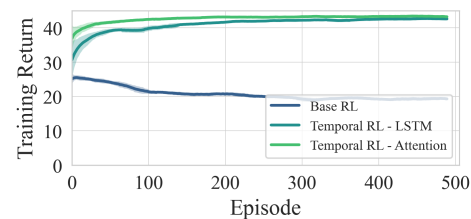
Case 1: Microgrid Control

Attention Temporal RL



Case 1: Microgrid Control

Returns and Losses of RL



Case 1: Microgrid Control

Cost

Method	Train Cost	Eval Cost
MILP with Uncertainty	180,792	184,843
Base RL (SAC)	170,585	170,929
Temporal RL - LSTM	148,149	150,169
Temporal RL - Attention	146,960	149,735
Deterministic Solution	143,669	146,671

Papers & Code

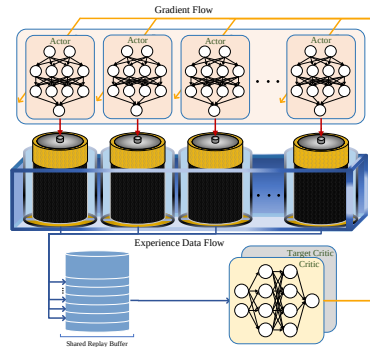
- 1 Heydarian Ardakani, et. al, "Embedding Temporal Awareness in Reinforcement Learning Models for Energy System Control". Full papers here:
<https://dl.acm.org/doi/full/10.1145/3679240.3734686>
 - 2 Heydarian Ardakani, et. al, "Attention-Based Temporal Reinforcement Learning for Energy System Control", Full paper here:
<https://ieeexplore.ieee.org/abstract/document/11305619>
- The code for Temporal RL could be found here:
<https://github.com/ES-Lab/ES-Temporal-RL>
 - The code for MILP and Base RL (SAC) could be found here
<https://github.com/ES-Lab/Edu-Kit>

Case 2: BMS

Problem and case study

Problem: Battery management system

- State: SOC, voltage, temperature, demand, price
- Action: charge/discharge
- Reward: cost



RL setup

State:

$$s_t = \left[SoC_i \quad v_i \quad T_i \quad \overline{SoC} \quad dev(SoC) \right]^T.$$

Critic:

$$\begin{cases} \mathbb{E}_\phi = \frac{1}{K} \sum_{i=1}^K \left(\hat{Q}_{\phi,t}^i - Q_{\phi^-,t}^i \right)^2, \\ Q_{\phi^-,t}^i = r_t + \gamma Q_{t+1}^i. \end{cases}$$

Actor:

$$\mathbb{E}(\theta) = -\frac{1}{K} \sum_{i=1}^K (Q_\phi(s_t, u_t) - \alpha \log \pi(s_t)).$$

Reward Function

Reward

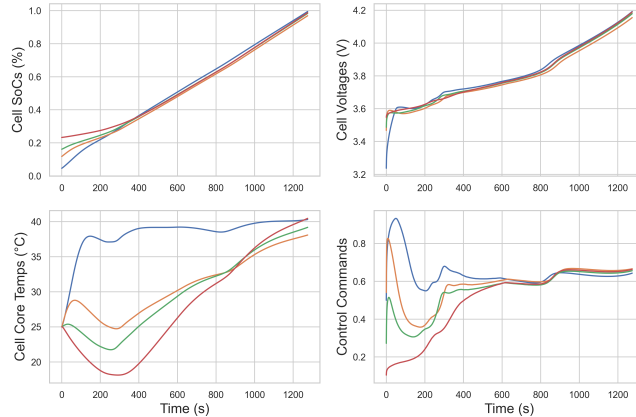
$$J_i = \sum_{t=0}^{\infty} J_{i,t} = \sum_{t=0}^{\infty} (C_{balance_i} + C_{SoC_i} + C_{constraints_i}).$$

The gradient descent algorithm is utilized to update the agent parameters. Leveraging the actor loss, the policy update rule is defined as:

$$\frac{\partial \mathbb{E}_{\text{actor}}}{\partial \theta} = - \left[\frac{\partial Q_{\phi}(s_t, u_t)}{\partial \theta} - \alpha \frac{\partial (\log \pi(a | s))}{\partial \theta} \right].$$

Case 2: BMS

Result



Result

Comparative analysis between RL, MPC and CCCV methods for BMS

Criteria	Scalable DRL	MPC	CCCV
Episode Cost	2839	2071	17861
Balancing Deviation	0.88%	0.13%	0.16%
Balancing Time	263	423	313
Full Charging Time	1554	1690	1490
Peak Temperature	36.99	40	51.88
Inference Time	2.1 ms	117.6 ms	0.9 ms

Paper & Code

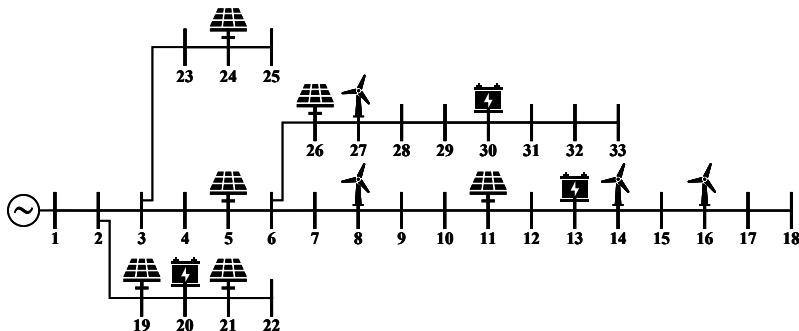
Heydarian Ardakani, et. al, "Scalable Charging Optimization of Battery Energy Storage Systems with Deep Reinforcement Learning,". Read the full paper here: <https://ieeexplore.ieee.org/abstract/document/10863295>

The code could be found here: <https://github.com/arbit3rr/Scalable-Charging>

Case 3: Power System

Problem and case study

Problem: The stochastic dynamic optimal power flow (SDOPF) problem in power systems considering nonlinear power flow constraints, uncertainties, and inter-temporal dynamics.



Objective

$$\min \sum_{t \in \mathcal{T}} C_t = \sum_{t \in \mathcal{T}} \left(\xi_t P_t^G + c^{\text{curt}} \sum_{i \in \mathcal{R}} (P_{i,t}^{r,\text{avail}} - P_{i,t}^r) \right)$$

- Unsafe exploration in power systems can violate voltage, thermal, or power limits.
- Safety must shape the learning process, not only the final policy.
- Policy updates are constrained by physical consequences.

Stochastic Dynamic Optimum Power Flow

$$P_{i,t} = 1_{\{i=0\}} P_t^{\text{slack}} + 1_{\{i \in \mathcal{R}\}} P_{i,t}^r + 1_{\{i \in \mathcal{B}\}} (P_{i,t}^{b,\text{dis}} - P_{i,t}^{b,\text{ch}}) - P_{i,t}^l,$$

$$Q_{i,t} = 1_{\{i=0\}} Q_t^{\text{slack}} + 1_{\{i \in \mathcal{R}\}} Q_{i,t}^r + 1_{\{i \in \mathcal{B}\}} (Q_{i,t}^{b,\text{dis}} - Q_{i,t}^{b,\text{ch}}) - Q_{i,t}^l,$$

$$P_{i,t} = \sum_{j \in \mathcal{N}_i} [g_{ij} |V_{i,t}|^2 - |V_{i,t}| |V_{j,t}| (g_{ij} \cos \delta_{ij,t} + b_{ij} \sin \delta_{ij,t})],$$

$$Q_{i,t} = \sum_{j \in \mathcal{N}_i} [-b_{ij} |V_{i,t}|^2 - |V_{i,t}| |V_{j,t}| (g_{ij} \sin \delta_{ij,t} - b_{ij} \cos \delta_{ij,t})],$$

$$(I_{ij,t})^2 = (g_{ij}^2 + b_{ij}^2) [(V_{i,t})^2 + (V_{j,t})^2 - 2V_{i,t} V_{j,t} \cos(\delta_{ij,t})],$$

$$\forall i \in \mathcal{N}, \forall (i,j) \in \mathcal{E}, \forall t \in \mathcal{T}.$$

RL Setup

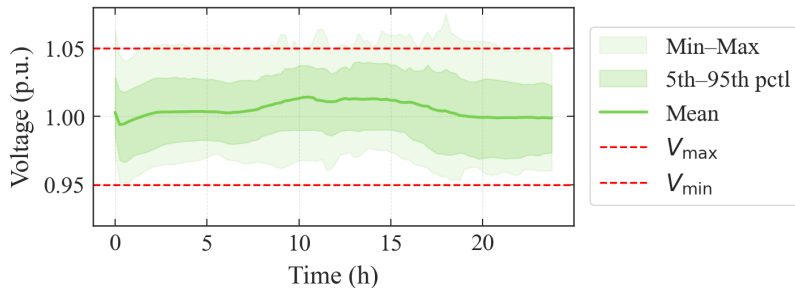
Graph-Based Constrained Markov Decision Process

$$s_{i,t} = \left[t, P_{i,t}^l, P_{i,t}^{\text{pv}}, P_{i,t}^{\text{wt}}, \text{SOC}_{i,t}, P_{i,t}, Q_{i,t}, V_{i,t}, \delta_{i,t} \right]^\top, \quad \forall i \in \mathcal{N}, \forall t \in \mathcal{T}$$

$$a_{i,t} = [U_{i,t}^P, U_{i,t}^Q]^\top, \quad \forall i \in \mathcal{R} \cup \mathcal{B}.$$

Results

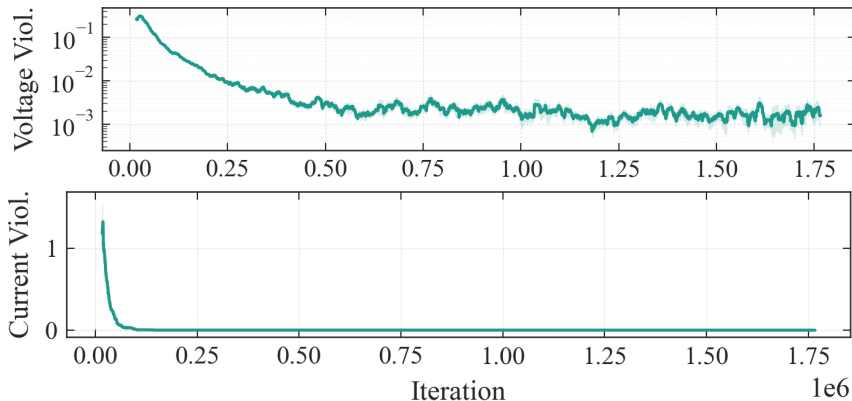
Bus voltage envelope across all test days



Case 3: Power System

Voltage and Current Violation

Cost of current and voltage violation during training



Conclusion

Which RL methods to choose?

Setting	Typical action space	Good first choice
Small discrete control	discrete	Q-learning / DQN
Continuous control	continuous	SAC / PPO
Logged data only	mixed	Offline RL
Strong model available	mixed	MPC or model-based RL
Hard safety constraints	mixed	Safe / constrained RL

Common failure modes of RL in energy systems

- **Reward misspecification:** a poorly designed reward can incentivize behavior that improves the numerical objective while degrading true system performance.
 - Reducing short-term cost at the expense of voltage violations, excessive switching, battery degradation, or poor long-term reliability.
- **Incomplete state representation:** if critical variables are omitted, such as network constraints, forecasts, hidden exogenous disturbances, device limits, or internal states of storage systems, the problem is no longer effectively Markov and the learned policy may appear unstable or inconsistent.
- **Ignoring physical structure:** purely data-driven learning without embedding grid physics, safety constraints, or operational logic can produce actions that are infeasible, unsafe, or non-interpretable, especially under rare but critical operating conditions.

Common failure modes of RL in energy systems

- **Poor generalization:** policies often perform well on the training distribution but not under forecast errors, topology changes, and renewable generation intermittency.
- **Unsafe exploration:** standard RL relies on trial-and-error interaction, but in energy systems, exploratory actions may violate operational limits, or trigger unacceptable behaviour in the system.
- **Simulation-to-reality gap:** policies trained in simplified simulators may exploit modeling inaccuracies and fail when deployed on real systems with delays, noise, unmodeled dynamics, communication constraints, and imperfect measurements.
- **Sample inefficiency and weak reproducibility:** many RL methods require large amounts of data, careful tuning, and significant computational effort, while results may remain sensitive to hyperparameters and implementation details.

To RL or Not?

RL is powerful, but it does not mean we should use it everywhere.

- The model is accurate, and optimization is tractable.
- Safe exploration is impossible.
- The problem is one-shot rather than sequential.
- The available data is biased or non-representative.

A critical perspective on RL in energy systems

RL is powerful for sequential decision problems, but in energy systems it is not a plug-and-play solution. Its success depends on careful problem formulation, constraint handling, and strong incorporation of domain knowledge.

A useful framework for RL in energy systems

1 Layer 1 - Decision

- policy, value, reward, optimization

2 Layer 2 - Information

- state design, forecasts, embeddings

3 Layer 3 - Physical system

- constraints, dynamics, and behavior

4 Layer 4 - Safety

- feasibility, robustness, certification, stability

A strong RL method addresses all four layers.

Final takeaway

- Reinforcement learning becomes useful in energy systems when it is treated not as a generic AI tool, but as a structured control methodology shaped by state design, temporal information, physics, safety, and deployment constraints.

**The real frontier is not applying RL to energy systems.
It is making RL credible enough for energy systems.**

Resources

- 1 Emma Brunskill, *Reinforcement Learning Course*, Stanford University.
- 2 Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*.
- 3 David Silver, *Reinforcement Learning Course*, University College London.
- 4 John Schulman and Pieter Abbeel, Deep Reinforcement Learning Course, University of California, Berkeley.

Special thanks to my PhD student, *Amirhossein Heydarian*, for helping with the GitHub repository.

Question

Questions?

Feel free to reach out to me: e.shirazi@utwente.nl